

The Alberta Buck - Publication and Deployment (v0.2)

What is published, why it is built the way it is, and what remains

Perry Kundert

2026-07-26 12:00:00

The BUCK codebase had one consumer: this checkout. Every entry point was a **make** target, every import path a `sys.path` shim, every binary artifact gitignored and rebuilt locally on demand.

The **Alberta Buck game** – a board projected on a TV, personal devices showing private net worth, real economic decisions played against real BUCK mechanics – is the forcing function that ends that. A game runs on someone else's machine.

Nine artifacts are now published and installable. Getting there was less about packaging than about **determinism**: an artifact you cannot reproduce is an artifact you cannot honestly publish, and the repository turned out to hold two independent sources of irreproducible binaries. Both are gone. This document records what is live, why the build is shaped the way it is, and what remains.

Two facts still govern everything. **Publishing is publishing**: a private GitHub repository and a public package are entirely compatible, but the tarball we upload contains the source, so the question is never "can we publish while private" – it is "which code are we ready to make public". And **the first upload is irreversible**: crates.io versions cannot be deleted, PyPI filenames cannot be reused, npm unpublish is a 72-hour window.

Contents

1	Status	4
2	The architecture, and why	4
2.1	Names: everything is <code>alberta-buck-*</code>	4
2.2	Licence: CAL-1.0 for the kernels, GPL-3.0 for contracts and sim	5
2.3	Versions: lockstep at major.minor, patch floats	6
2.4	One compiler for everything we own	7
2.5	Third-party artifacts come from their publishers	7
2.6	The contracts have no addresses	8
3	Mechanics worth not rediscovering	9
3.1	The first publish is always manual	9
3.2	Pending publishers are unique on the config, not the project .	9
3.3	Build outputs are not published by default	10
3.4	Dependencies must be pinned to be reproducible	10
4	The public-code boundary	10
4.1	D1: the sim's source goes public [settled 2026-07-26]	10
5	Remaining decisions	11
5.1	D2: where the game's authority lives	11
6	The way forward	11
6.1	P2.5b: <code>alberta-buck-contracts</code> on PyPI [done 2026-07-26] .	11
6.2	P3: <code>alberta-buck-core</code> on PyPI, and the pure/binary split [done 2026-07-27]	12
6.3	P4: <code>alberta-buck-kernel</code> wheels	13
6.3.1	The macOS platform tag is set at the link, not at pack- aging	14
6.3.2	The staging import check is not the gate, and must not act like one	14
6.3.3	An empty environment variable is not an unset one . .	15
6.3.4	JSON is UTF-8; <code>read_text()</code> is whatever the locale says	15
6.3.5	Wheels only – no sdist	16
6.3.6	What gates a wheel	16
6.4	P5: <code>alberta-buck</code> on PyPI	16
6.5	P6: what the game adds	17

6.6	P7: making the repository public	17
7	Risks	18

1 Status

Published, and each verified by installing from the registry into an empty project outside this checkout:

artifact	registry	version	licence
alberta-buck-math	crates.io	0.1.0	CAL-1.0
alberta-buck-identity	crates.io	0.1.0	CAL-1.0
alberta-buck-registry	crates.io	0.1.0	CAL-1.0
alberta-buck-wallet	crates.io	0.1.0	CAL-1.0
alberta-buck-kernel	npm	0.1.1	CAL-1.0
alberta-buck-core	npm	0.1.1	CAL-1.0
alberta-buck-contracts	npm	0.1.0	GPL-3.0-or-later
alberta-buck-contracts	PyPI	0.1.0	GPL-3.0-or-later
alberta-buck-core	PyPI	0.1.0	CAL-1.0

Remaining:

#	artifact / step	registry	blocked on
4	alberta-buck-kernel	PyPI	nothing – tag <code>py-kernel-v0.1.0</code>
5	alberta-buck	PyPI	dependency hygiene
6	alberta-buck-game + zomes	npm	D2 – where authority lives
7	repository public	GitHub	a deliberate decision

In place: `release-crates.yml`, `release-npm.yml` and `release-pypi.yml` (all trusted publishing, no stored tokens), `LICENSE / core/LICENSE / LICENSING.md / NOTICE`, a Rust toolchain and `binaryen` in the flake, `Cargo.lock` and `foundry.lock` committed. The repository is pushed to `github.com/alberta-buck/alberta-buck` (private), and `release-pypi.yml` has run green – the PyPI row above is the first artifact this repository published without a human holding a token.

2 The architecture, and why

Six decisions shape everything above. They are settled; the reasoning is recorded because the *consequences* keep surfacing, not to reopen them.

2.1 Names: everything is alberta-buck-*

Every published distribution carries the project prefix, on every registry. The same name on two registries names that language’s build of the same thing

– `alberta-buck-core` is the Python platform on PyPI and the JS platform on npm, deliberately.

Import names do **not** change: `import buck_core`, use `buck_math::`, `import { Session } from "alberta-buck-core"`. Distribution name and import name are different things (`python-slip39` imports as `slip39`). For the crates this needs an explicit `[lib] name`, since cargo would otherwise derive `alberta_buck_math` from the package name. Directory names do not change either: `core/rust/buck-math` stays.

The cost of unscoped names, paid twice: the first publish of a **new** package needs a broadly-scoped token, because a package that does not yet exist cannot appear in a token’s package picker. With `@alberta-buck/*` a token could have been scoped to the scope. The ergonomics of npm i `alberta-buck-core` were judged worth it; the `@alberta-buck` scope is reserved anyway, against typosquatting.

2.2 Licence: CAL-1.0 for the kernels, GPL-3.0 for contracts and sim

The rule: start restrictive and relax deliberately. A licence can be loosened later, or granted separately to a counterparty; it can never be tightened on a version already published.

The goal is that an organization intending to build a *liberty-restricting* system – custodial control of other people’s keys, freezable balances, users denied their own records – is at minimum operating out of licence. No OSI-approved licence can do this by restricting *purpose*: OSD 6 forbids discrimination against fields of endeavour.

The **Cryptographic Autonomy License** (CAL-1.0, OSI-approved 2020, drafted for Holochain) restricts *architecture* instead, which is both enforceable and self-executing:

section	obligation
4.1	provide Recipients the Source Code, or unrestricted network access to it
4.1.2	modifications go out under CAL-1.0 or a compatible licence
4.1.3	a 90-day embargo is permitted for coordinated vulnerability disclosure
4.2.1	give each Recipient a no-charge copy of their own User Data
4.2.2	no cryptographic or technological measure limiting a Recipient’s access to functionality
4.2.3	no contractual restriction on a Recipient exercising the same permissions independently
4.5	Combined Work Exception: per-file opt-in permitting combination into a larger work under its own licence

A system that holds users’ keys, can freeze their balances, or withholds

their records **cannot comply with 4.2 while using CAL-covered code**. It is out of licence the day it ships – no intent need be proven, and no field of endeavour is discriminated against.

Applied to the kernels, the bindings and both platform libraries, because that is where the leverage is: anyone building a control system needs the identity, wallet and Notes kernels. CAL anywhere else would be decorative. The contracts and `alberta_buck` are GPL-3.0-or-later, which all 49 `src/*.sol` files already declared.

Consequences to plan for:

- **The Combined Work Exception is the partner path, and it is one-way.** Marking a file CWE cannot be undone for a published version. Mark it narrowly and late – at the binding surface, if a real integrator needs it – never across the kernels wholesale. **No file carries it today.**
- **Dual licensing is the enforcement lever**, and works only while the copyright is held in one place. An organization that cannot comply must come and ask, and may be declined. This requires a **CLA** the moment the repository is public and outside contributions arrive.
- **Expect friction.** CAL is young and thinly litigated; counsel will read it rather than recognize it. That is the instrument working, and a real cost against adoption.
- **The licence governs code, not ideas.** The papers are public; the mechanism can be reimplemented. What CAL buys is that it cannot be done *with our code* while denying users their autonomy.

2.3 Versions: lockstep at major.minor, patch floats

The kernels are one artifact compiled three ways, so the bit-identical guarantee must be anchored to a version – but it is a claim about *semantics*, and only major.minor can change semantics.

- `alberta-buck-math 0.1.*` in Rust, JS and Python compute identically. The cross-language vector suites enforce this, and may assume it.
- A patch release in one language does **not** oblige the others. The JS packages went to 0.1.1 for a `wasm-opt` size pass that did not touch a line of Rust; dragging four identical crates along would have been ceremony, not information.

- A major or minor bump is a cross-language event. It lands everywhere, or the guarantee stops being true.

Inter-crate requirements stay caret ("0.1"), and `release-crates.yml` guards that they track the workspace major.minor – a stale "0.1" would silently resolve to the **previous** release once the workspace moved to 0.2.0, publishing a crate built against its own old sibling.

Tags are per-registry: `rs-v*`, `js-v*`. One `v0.1.0` firing all three breaks the moment an artifact is unchanged, since crates.io rejects a republished version.

PyPI goes one level finer – `py-contracts-v*`, `py-core-v*`, `py-kernel-v*`, `py-sim-v*` – because its four distributions are genuinely independent packages rather than one workspace released together. A shared `py-v*` would have to either force all four to the same version, contradicting the patch-floats rule above, or abandon the tag-matches-version gate. Neither is worth saving three tag prefixes.

2.4 One compiler for everything we own

The decision this repository fought hardest, worth the detail because the failure mode was invisible.

Our Solidity sources are ranged (`~0.8.20`), so auto-detection had a choice – and made it differently depending on which target ran. `make build` passed `--use 0.8.28`; the bare `forge build` inside `build-uniswap-artifacts` did not, because that one invocation also spanned the Uniswap V2 trigger pinned at `=0.5.16`, and no single `--use` satisfies both. Both wrote `out/`. After a full `make test`, `out/` held **five compiler versions**, with `Buck.json` at 0.8.31 – the very version the pin exists to avoid (an IR codegen bug, "Modifiers not implemented yet").

Which compiler produced a published artifact would have depended on which `make` target ran last.

Now `[profile.default]` declares `solc = "0.8.28"` and `auto_detect_solc = false`. The pin is configuration, not a command-line flag that applies only where someone remembered it. Moving it changes every byte we publish, so it is a version event.

2.5 Third-party artifacts come from their publishers

The Uniswap contracts our worlds deploy are pinned to pre-0.8 pragmas: `v2-core =0.5.16`, `v2-periphery =0.6.6`, `v3-core =0.7.6`. Building them here meant three extra compilers – and foundry on arm64 macOS cannot resolve

any of them, failing with "No solc version exists that matches the version requirement" even with the official binaries installed in `~/svm`.

The artifacts that lived in `out/` were **stale leftovers** from a toolchain no longer present here, surviving only because `v2-patch-init-code-hash` guards its rebuild behind `test -f`. The suite's 529 passes were running against contracts **no current toolchain could reproduce**. A fresh clone, or CI, would simply have failed.

Uniswap publishes those artifacts themselves. `scripts/stage-uniswap.mjs` stages them into `out/` in the forge layout that `vm.deployCode` and both artifact loaders already expect, from exactly-pinned npm packages. This deleted three compilers, two foundry profiles, both compile-trigger directories and the `FOUNDRY_VIA_IR` juggling – and it is exactly what we tell consumers of `alberta-buck-contracts` to do.

Two things this surfaced:

- **The official bytecode is canonical.** `UniswapV2Pair` hashes to `0x96e8ac42...`, the canonical mainnet init-code-hash, so pool addresses now derive exactly as on mainnet and `v2-patch-init-code-hash` has become a no-op that verifies rather than patches.
- **v2's combined-json omits the 0x prefix on bytecode; v3's hard-hat output includes it.** Foundry tolerates both, so the forge suite passed either way – but `viem/tevm` reject unprefixed hex, and the JS worlds failed with "deploy WETH9 reverted" until the normalizer forced the prefix. Precisely the class of divergence that two artifact sources conceal.

2.6 The contracts have no addresses

Nothing is deployed at a fixed address, on any chain. Every BUCK world – Python sim, JS sim, browser demos, tests – deploys fresh into its own EVM and learns addresses from the receipts. So `alberta-buck-contracts` ships (`abi`, `bytecode`) pairs, not a deployment registry. `deployments.json` is empty; the slot exists so publishing real addresses later is not a breaking change to the package's shape.

Only the eight contracts we own are published – an allowlist, not a wildcard, because a wildcard over `out/` would sweep in interface stubs, test fixtures and the vendored Uniswap implementations that share the directory. Asking either package for a third-party contract raises an error naming the package to install instead.

`compiler.json` records the exact solc version, optimizer settings, EVM version, git commit and a sha256 over the bundle; the build asserts every artifact was produced by the pinned compiler.

3 Mechanics worth not rediscovering

3.1 The first publish is always manual

Every registry requires a package to **exist** before a trusted publisher can be configured for it:

- **crates.io** – `rust-lang/crates-io-auth-action@v1` needs the crate present; the first publish used a scoped token.
- **npm** – the same, with an open request for OIDC-first-publish ([npm/cli#8544](#)). Needs `npm >= 11.5.1` and `Node >= 22.14.0`; `ubuntu-latest` ships an older `npm`, so the workflow upgrades it first.
- **PyPI** is the exception: **pending publishers** let a project be created by its first OIDC upload, so PyPI can be zero-token from the start – once the workflow exists and the repository is pushed.

3.2 Pending publishers are unique on the config, not the project

The natural reading of PyPI's form – four projects, same owner, same repository, same workflow, differing only in project name – is rejected on the second registration:

A pending trusted publisher matching this configuration has already been registered for a different project name.

Warehouse enforces a database constraint on (`owner`, `repo`, `workflow`, `environment`) and deliberately leaves `project_name` out of it. The reason is sound: the OIDC claim is **all** PyPI has to decide which project to mint, so two pending publishers with identical configs would be ambiguous. The discriminator has to be something the claim carries, and the cheap one is the **environment**:

project	environment
alberta-buck-contracts	pypi
alberta-buck-core	pypi
alberta-buck-kernel	pypi-kernel
alberta-buck	pypi-sim

The constraint binds **pending** publishers only – established ones may share a config freely, which is why **contracts** and **core** coexist on **pypi** (**contracts** was established before **core** was registered). There is no reason to consolidate them afterwards: separate environments allow per-package protection rules.

npm generates provenance attestations only for **public** repositories. While this repository is private, publishes succeed but carry no provenance, and **--provenance** must not be passed: it fails outright rather than degrading.

3.3 Build outputs are not published by default

npm falls back to **.gitignore** when there is no **.npmignore**, and honours it **over** the files allowlist. **wasm-pack** writes a **.gitignore** of **"*"** into its out-dir, so the first **npm pack --dry-run** of the kernel produced a tarball with a **README**, a **package.json**, and **no wasm at all**. The build removes it, and **release-npm.yml** asserts four **.wasm** files in the tarball rather than trusting that it did.

3.4 Dependencies must be pinned to be reproducible

make install fetched HEAD of five repositories and **foundry.lock** was git-ignored. **OpenZeppelin** compiles **into** our **contracts**, so a 5.6.1 -> 5.7 bump would have changed published bytecode with no commit to show for it. Every dependency now pins an exact tag; **foundry.lock** and **Cargo.lock** are committed.

4 The public-code boundary

4.1 D1: the sim's source goes public [settled 2026-07-26]

Publishing a Python package publishes its source: an sdist is a tarball of the package directory, a wheel a zip of **.py** files. There is no meaningful "binary only" for Python. So publishing **alberta_buck** publishes the sim, the equilibrium findings (**EQUILIBRIUM.md**, the sweep vectors, **rebalance_policy.py**) and the quote sources.

Settled: yes, publish it. P5 is unblocked; every planned PyPI distribution can now ship.

What this does not decide: the repository itself stays private until P7 is taken deliberately, and the **papers** keep their own schedule. Publishing the package makes the code readable; it does not make the writing done.

5 Remaining decisions

5.1 D2: where the game's authority lives

Three places the mechanics could be authoritative:

1. **Python server** (`alberta_buck.sim.server`: `pyrevm` worlds at interactive speed, websockets, multiple clients). Highest fidelity – the real contracts, the real EVM. Requires a host, and `pyrevm` has no linux-aarch64 wheels and none for macOS past cp312, so the target must be `x86_64` or build from source.
2. **Browser EVM** (`Tevm`, per player or per board). Local-first, no server, already exercised by `core/js/test/*.tevm.test.js` – but `Tevm` is the slow backend and `farmsim.mjs` OOMs past ~6 simulated months.
3. **Kernel-only** – the game computes on `alberta-buck-math` and `alberta-buck-wallet` directly, with no EVM anywhere. The kernels exist precisely because they are bit-identical to the contracts; a game does not need EVM semantics, it needs the arithmetic to be right, and periodic vector checks keep it honest. Runs at native speed via `wasm` in a browser, and unchanged in a Holochain zome.

Recommendation: (3), with (1) as the fidelity anchor. It is the only option that is local-first – the point of choosing Holochain – the only one that runs on a phone, and it makes the `wasm` kernel rather than a server the game's critical dependency. Under that choice PyPI is the research distribution, not the game's runtime.

This does not make the contract artifacts optional: they build the fidelity anchor, the vector generator compiles them, and any "show me this on a real chain" moment needs them.

6 The way forward

6.1 P2.5b: `alberta-buck-contracts` on PyPI [done 2026-07-26]

Shipped 0.1.0 with no token ever created: a pending publisher, and the first OIDC upload minted the project. The bootstrap that `npm` and `crates.io` each needed a scoped token for, PyPI did not need at all.

Accepted: `pip install alberta-buck-contracts` in a clean venv outside any checkout returns Buck's ABI and 16,157 bytes of bytecode.

6.2 P3: alberta-buck-core on PyPI, and the pure/binary split [done 2026-07-27]

Shipped 0.1.0 as a pure py3-none-any wheel. The reasoning below is kept because the shape it produced is what P4 and P5 are built on.

core/python/buck_core/ held four .so files, three of them **byte-identical copies of one dylib** (buck_identity.so, buck_wallet.so, buck_registry.so: 1,491,824 bytes each – one cdylib with three #[pymodule] entry points). Right for the Makefile, wrong for a wheel: it triples 1.5 MB for nothing and makes alberta-buck-core platform-specific, dragging the whole Rust wheel matrix onto the critical path of pip install alberta-buck.

Split along the seam the runtime already has:

alberta-buck-core	pure Python, py3-none-any. session, journal, artifacts. Hard dependency of alberta-buck. No Rust in CI.
alberta-buck-kernel	binary, abi3-py311 per platform. Optional: 'pip install alberta-buck[kernel]', selected at runtime by BUCK_IDENTITY_BACKEND, as today.

Keep import buck_core.buck_identity working with a shim re-exporting from the kernel package when installed, so no calling code changes.

Within the kernel wheel, ship the dylib **once**: Python's extension loader takes the module name and the file separately, and looks up PyInit_<name> in whatever file it is given.

```
# buck_kernel/buck_wallet.py -- one .so, three modules
import sys, importlib.util
from importlib.machinery import ExtensionFileLoader
from pathlib import Path

_so = str(Path(__file__).with_name("_kernel.abi3.so"))
_spec = importlib.util.spec_from_file_location(
    __name__, _so, loader=ExtensionFileLoader(__name__, _so))
_mod = importlib.util.module_from_spec(_spec)
_spec.loader.exec_module(_mod)
sys.modules[__name__] = _mod
```

4.9 MB of wheel becomes 1.9 MB. The fallback – ship the dylib three times – is wasteful, not broken.

The core suite (`core/python/tests`) is deliberately **not** the release gate: its vector tests need a built kernel and its session tests a live EVM, neither of which this wheel contains or requires. Gating on it would make a pure-Python release depend on the Rust toolchain – exactly the coupling the split exists to remove. The job gates on what the wheel actually promises:

- the wheel is `py3-none-any` and carries no `.so`, `.pyd` or `.dylib`. The four `.so` files still sit beside the shims in a checkout, so this is a live risk, not a theoretical one – a future `package-data` entry would ship them and produce a platform wheel that installs on the wrong host and fails at import;
- installed from the wheel into a clean venv **outside any checkout**, `load_artifact("Buck")` returns 46 ABI entries and 16,157 bytes from the installed `alberta-buck-contracts`. Run from a temp cwd, because `repo_root()` walks up from both `__file__` and the cwd – testing inside the checkout would prove nothing;
- with no kernel installed, each `buck_core.buck_*` shim raises `ImportError` – precisely what `alberta_buck.wallet._kernel` catches to fall back to pure Python. Any other failure would break the selector.

6.3 P4: alberta-buck-kernel wheels

The only phase with real engineering. `abi3-py311` was already set in both binding crates, which is the large simplification: **one wheel per platform**, not one per Python version, and it keeps working on 3.13, 3.14 and beyond with no rebuild.

Targets: `macOS arm64 + x86_64`, `manylinux_2_28 x86_64 + aarch64`. Each has a native GitHub runner (`macos-13` is retired, but `macos-15-intel` replaces it), so **nothing is cross-compiled** and every wheel is tested on the architecture it was built for. **linux-aarch64 is included** – it is the gap that bites `pyrevm` today, and ARM servers are where a cheap game backend lives.

Windows is deferred, on priority rather than difficulty: on its first run the `.pyd` built, loaded and passed 32 of the 39 tests. Re-enabling is a matrix entry with `venv_bin`: `Scripts` plus `win_amd64` in the publish job's expected set. Until then Windows gets `pip install alberta-buck` on the pure-Python path, which is the supported configuration everywhere anyway.

Not maturin, as this document previously proposed. `maturin` builds one extension module per wheel and we ship two cdylibs in one – `_math` (zero-

dependency arithmetic) and `_kernel` (identity + wallet + registry sharing a single object). Adopting it would have meant collapsing that structure to suit the tool. `scripts/stage-kernel.py` does the part maturin was wanted for – it knows each platform’s cdylib filename and staged suffix – and `auditwheel` inside the manylinux image does the rest.

Linux builds inside `quay.io/pypa/manylinux_2_28_*`, **not on the runner**. A wheel’s minimum glibc is whatever it linked against; `ubuntu-latest` is 2.39, which almost nothing outside a current Ubuntu can load.

6.3.1 The macOS platform tag is set at the link, not at packaging

The trap worth the most words, because every instinct about it is wrong. A macOS wheel’s platform tag does not come from `MACOSX_DEPLOYMENT_TARGET`, and not from the packaging step: the `wheel` project reads the Mach-O `LC_BUILD_VERSION` out of every object in the wheel and takes the **greater** of that and the building interpreter’s `sysconfig` platform.

Both halves have to be pinned, and each was found by a wheel coming out wrong:

- `-C link-arg=-mmacosx-version-min=11.0=` in `core/rust/.cargo/config.toml`. Setting `MACOSX_DEPLOYMENT_TARGET` in the environment was **ignored** – nix supplies its own SDK default, and the object still came out `minos 14.0`. Pinning at the link works regardless of toolchain.
- `_PYTHON_HOST_PLATFORM` in the workflow, for the interpreter’s half.

Unpinned on a macOS 26 runner the result is `macosx_26_0_arm64`: a wheel that uploads without complaint and installs on essentially nothing. `scripts/check-kernel-wheel.py --expect-platform` asserts the exact tag per matrix entry, so either pin coming loose fails the build instead.

6.3.2 The staging import check is not the gate, and must not act like one

On the second run of an unchanged commit, `macos-arm64` failed the post-stage import with dyld reporting *slice is not valid mach-o file* – the error an unreadable object gives. The object was **byte-for-byte identical** to the one that had loaded cleanly the run before: 483,696 bytes, same runner image (`macos-26-arm64 20260720.0258.1`), same rustc 1.97.1, same `host macosx-10.9-universal2`. The bytes were not the variable.

The root cause is unproven. What the check now does instead of guessing:

- each import runs in a **subprocess**, and gets **one retry**;
- the staged object is **fsync'd** after copy, since the very next thing that touches it is a **dlopen** and the failure mode is exactly what a partially visible file produces;
- on a second failure, **describe()** prints the object's real architecture (**file, lipo -archs**) and the interpreter's, so the next occurrence is diagnosed rather than theorised about.

This is proportionate because the check is **redundant**. It is an early convenience – catch a missing `PyInit_` before packaging – and the actual gate is downstream: the wheel is installed and the conformance vectors run against it. A redundant check that can fail spuriously is worse than no check; one that retries and then explains itself is worth keeping.

6.3.3 An empty environment variable is not an unset one

`_PYTHON_HOST_PLATFORM` was passed through the job's `env:` block, so on the matrix legs that do not define it GitHub set it to `""` rather than leaving it out. CPython's `sysconfig.get_platform()` tests

```
if "_PYTHON_HOST_PLATFORM" in os.environ:      # membership, NOT truthiness
    return os.environ["_PYTHON_HOST_PLATFORM"]
```

so it returned the empty string, and both Linux legs died on `Bad wheel filename 'alberta-buck-kernel-0.1.0-cp311-abi3-.whl'`. The variable is now exported inside the step only when it has a value. Worth remembering generally: a workflow `env:` key whose value is an undefined matrix field is *set and empty*, which is a different thing from absent to any code that checks membership.

6.3.4 JSON is UTF-8; `read_text()` is whatever the locale says

`Path.read_text()` with no `encoding` uses the platform's preferred codec. On Linux and macOS that is UTF-8 and the bug is invisible; on Windows it is cp1252, and the kernel vector files contain non-ASCII (`\xe6\x9d\xe`, a CJK name in a canonical-JSON test case). Every vector fixture raised `UnicodeDecodeError` at setup.

RFC 8259 makes JSON UTF-8 by definition, so all ten call sites under `core/` and `scripts/` now say `read_text(encoding="utf-8")=`. Two of them were in the **published** `alberta-buck-core-artifacts.py` and `session.py`

– where a Windows user loading a contract artifact would have hit the same failure with no test to catch it.

6.3.5 Wheels only – no sdist

The kernel sdist is a hazard, not a fallback. Its contents are `buck_kernel/` **including whatever .so the build host produced**, and none of the Rust that made it: published, pip on an unmatched platform would download it, "build" it, and install a foreign binary. With wheels alone that case is a clean *no matching distribution*, and `pip install alberta-buck` without `[kernel]` still works everywhere. The check is enforced in both the matrix job and the publish job.

6.3.6 What gates a wheel

The vector suite is the kernel's whole claim – bit-identical agreement with the pure-Python spec – so each wheel runs it on its own platform, installed, with `alberta-buck-core` from PyPI in front of it.

That suite alone would be worthless as a gate: its tests are guarded by `pytest.importorskip`, so a kernel that fails to load makes them **skip** and the run passes green having proven nothing. `check-kernel-wheel.py --installed` runs first and asserts all four modules resolved to compiled objects inside the installed `buck_kernel`. Verified both ways: 39 passed against the wheel, and the preflight fails with *No module named 'buck_kernel'* when the kernel is absent.

6.4 P5: alberta-buck on PyPI

Blocked on P3 (hard `buck_core` import) and D1. The contract blocker is cleared: `load_artifact` falls back to the installed `alberta-buck-contracts` when no `foundry.toml` is reachable, with a repo checkout's `out/` still winning so a rebuild takes effect immediately.

Remaining:

1. **Dependency hygiene.** `matplotlib` (plots only) -> `[plots]`; `websockets` (`sim/server.py` only) -> `[server]`; `pyrevm` (`sim/pyrevm_backend.py` only) -> `[fast]`. `pyrevm` especially: as a hard dependency it makes `pip install alberta-buck` **fail to build** on linux-aarch64 and on macOS with Python 3.13+, because upstream ships no wheels there. As an extra it is opt-in, and its absence degrades to the anvil backend, which the code already does.

2. Add `alberta-buck-core` and `alberta-buck-contracts` to dependencies, `alberta-buck-kernel` as `[kernel]`, then **delete the `sys.path shim`** in `alberta_buck/__init__.py` – its comment already anticipates this.
3. Align `requires-python` to `>= 3.11` (currently 3.10).
4. Dynamic version from `alberta_buck/version.py`, plus the tag-matches-version CI guard.

6.5 P6: what the game adds

Under D2/(3):

board (projected)		personal device		shared state
-----		-----		-----
<code>alberta-buck-game</code>	<code><-></code>	<code>alberta-buck-game</code>	<code><-></code>	Holochain <code>hApp</code>
<code>alberta-buck-core</code>		<code>alberta-buck-core</code>		(zomes: <code>Rust</code> ,
<code>alberta-buck-kernel</code>		<code>alberta-buck-kernel</code>		<code>alberta-buck-identity</code> ,
(<code>public: prices, pools,</code>		(<code>private: net worth,</code>		<code>alberta-buck-wallet</code>
<code>events, narrative</code>)		<code>holdings, decisions</code>)		from <code>crates.io</code>)

- `alberta-buck-game` (npm) – rules, turn structure, the narrative price engine (trade wars, embargoes, a mill fire), the mortgage-versus-issuance decision surface. The economics live here; the monetary arithmetic stays in the kernel.
- **Holochain zomes** (Rust, distributed as a `.happ` bundle rather than via a registry), depending on `alberta-buck-identity` and `alberta-buck-wallet` from `crates.io` – the concrete reason the crates went first. The kernels are `std` today; the `no_std` flip is mechanical if a zome needs it.
- **The narrative price generator** is wanted by both the game (JS) and the research sim (Python). If it must agree across both it belongs in the kernel with vectors; if not, keep two simple implementations and say so out loud.

6.6 P7: making the repository public

Independent of everything above. Checklist:

1. Re-run the secrets audit over full history (`.env` is clean and has never been committed; re-verify, and check `foundry.lock`, RPC URLs in committed JSON, and `.claude/`).

2. Decide about the 216 MB of history. Rewriting it – dropping the large circom power tables and PDFs – is a one-time option that disappears the moment the repository is public and forked.
3. `README.org` as a front door; `CONTRIBUTING`; `SECURITY.md` (a monetary system needs a disclosure address); the CLA that dual licensing requires.
4. Branch protection on `master`, requiring the CI gate.
5. Only then do npm provenance attestations begin to be generated.
6. The Solidity *sources* become public here; `alberta-buck-contracts` can then carry a source-verification link.

7 Risks

- **Publishing precedes the papers** (D1). Source on PyPI is source in public. Sequence deliberately.
- **Irreversibility.** crates.io yanks but never deletes; PyPI never re-uses a filename; npm unpublish is 72 hours. Rehearse on TestPyPI, use `--dry-run`, read `cargo package --list` before every first publish.
- **Unaudited contracts in a public registry.** Mitigated by prominent labelling, 0.x versions and an empty `deployments.json` – but someone will eventually deploy them anyway.
- **Binary wheel maintenance** is a standing tax: every new CPython, every new manylinux baseline. abi3 removes the per-version half; the per-platform half remains. Mitigated by `alberta-buck-kernel` being optional by construction.
- **Version skew across registries.** A JS game on kernel 0.2 talking to a Python server on kernel 0.1 is a bit-identity violation no test here would catch. Mitigated by lockstep at major.minor, and by a cross-registry vector suite each language runs against its *installed* artifact rather than its checkout.
- **Contract/kernel drift.** The kernels are bit-identical to the contracts *at a commit*. On independent version streams nothing enforces the pairing. `compiler.json` records the commit; the kernel crates should record the contract version their vectors came from, and assert it.

- **CAL is unfamiliar.** Counsel will read it rather than recognize it, and some will decline on that alone. Accepted deliberately – the friction is the instrument – but it makes dual licensing, and the CLA, part of the operating cost.
- **Third-party artifact trust.** We ship no Uniswap bytecode, but our worlds execute theirs from npm. Pinned exactly and lockfiled; the init-code-hash check is a standing canary that the bytecode is what mainnet derives from.
- **npm supply chain.** Trusted publishing, the `files` allowlist, and `npm publish --dry-run` in CI. No classic tokens, ever.
- **Holochain maturity** – the D2/(3) recommendation deliberately keeps the game’s monetary correctness independent of it.